Slime’s Space Travel

～開発要点書～

2026.11.5
日下　拓海

目次

1. 基本情報
2. 制作背景・コンセプト
3. ゲームの特徴
4. 技術概要
5. 開発用エディタ・制作支援機能
6. システム構成・設計について
7. 技術的に工夫した点・問題解決
8. ビルド・配布
9. まとめ

1. 基本情報

1-1. 作品情報

ゲームタイトル: Slime’s Space Travel

ジャンル: ステージクリア型 3D アクションゲーム

対応プラットフォーム: Windows / macOS

主要技術: C++20 / OpenGL / Bullet Physics / Assimp / SDL2 / yaml-cpp / Dear ImGui など

※ Unity、Unreal Engine などの汎用ゲームエンジンは使用していません。

制作期間: 2026.3 - 2026.11

※ 現在もプレイヤーからフィードバックをいただき、調整・改善を継続しています。

所要時間数の概数: 690 時間 ※ 開発時間を記録した範囲での概算です。

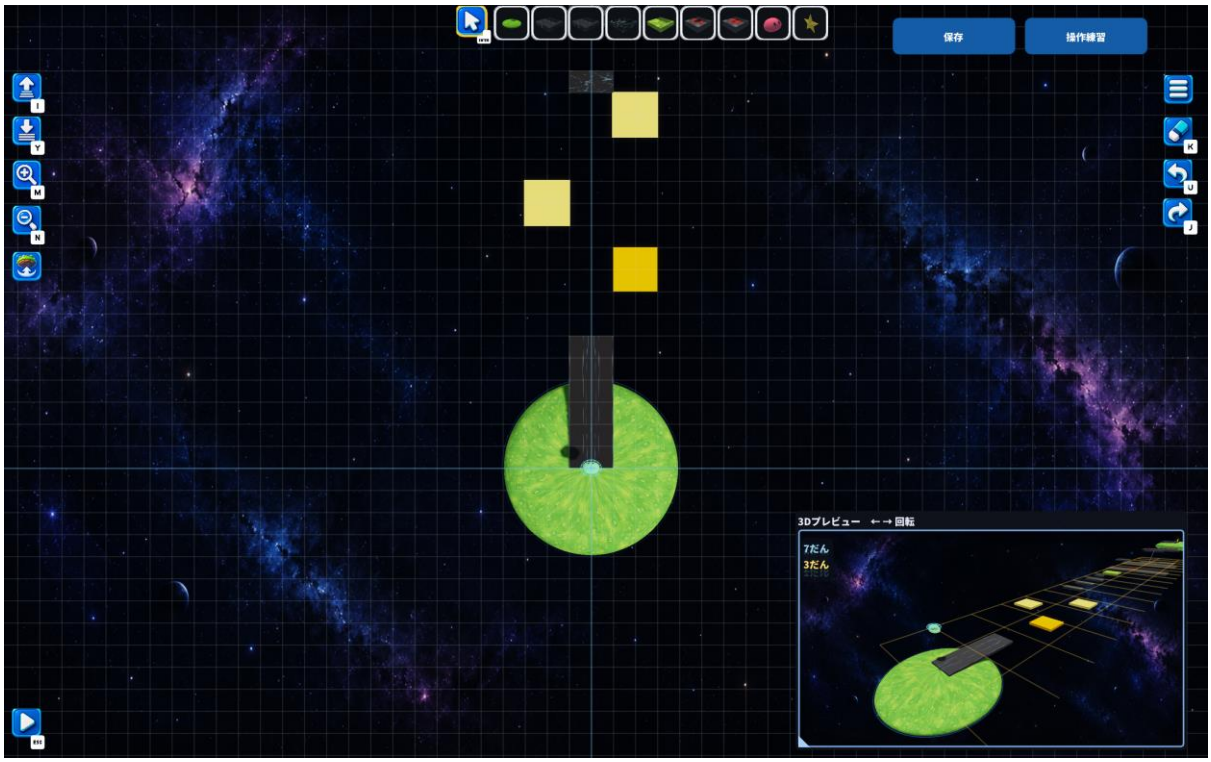
制作人数 : 1 人

担当範囲: ゲーム設計、全プログラム実装、ステージ制作、UI・デバッグツール・ステージ作成機能の実装、一部モデル制作など、開発全体を担当しました。

1-2. ゲーム概要

本作は、球体や楕円状の惑星を 360 度探索しながら、敵との戦闘や分身を使ったギミックを攻略して進むステージクリア型 3D アクションゲームです。 惑星の表・側面・裏側まで移動できる立体的なステージを舞台に、さまざまな惑星を冒険してゴールを目指します。





1-3. 主な技術的特徴

本作では、C++ / OpenGL を用いて 3D ゲームの主要システムを実装するとともに、本作独自のゲームシステムや開発支援機能を実装しています。特に以下の 3 点に力を入れました。

① 球体・楕円体を 360 度移動できる独自の接地・重力制御

Bullet Physics による Ray Test と地面法線を利用し、惑星の表・側面・裏側や斜面、足場に沿って移動できる仕組みを実装しています。楕円体では、各軸の半径から表面位置・法線を求める処理も独自に実装しています。

② 2D 操作で 3D ステージを作成できる UGC システム

3D 座標を直接操作するのではなく、水平方向を 2D グリッド、高さを「だん」として管理することで、立体的なステージを分かりやすい操作で作成できる仕組みを実装しています。作成中は 3D プレビューで確認でき、そのままプレイテストも可能です。

③ ゲーム制作そのものを効率化する開発用エディタ

ステージや UI の直接編集、パラメータ調整、アセット確認、GPU 負荷計測などをゲーム内から行える開発環境を実装しています。さらに、C++コード変更時には編

集状態を保存したままビルド・再起動できる仕組みも用意しています。

1-4. 簡易自己紹介

開発者名：日下　拓海

所属：弘前大学理工学部電子情報工学科 3 年

ゲーム業界・開発への想い：「ゲームが苦手」と感じる人を 1 人でも減らし、家族や友人と一緒にゲームを楽しめる体験をより多くの人に届けたいと考えています。

　そのため、初心者や子どもでも迷わず遊べる分かりやすさと、ゲームに慣れた人も楽しめる遊びの深さを両立したゲーム作りを目指しています。

　本作でも、迷いにくいステージ構成、2 人プレイ、らくらくスタイルなどを通して、この考えを形にしています。

1-5. プレイ映像

本作のプレイ映像です。

https://www.youtube.com/watch?v=VCZb3pjgQlE



2. 制作背景・コンセプト

2-1. 制作背景

友人と一緒に、大量の敵を相手にするアクションゲームを遊んでいた時の経験が、本作を制作するきっかけになりました。その友人は、ストーリー進行に直接関係のない敵を長時間倒し続けていました。理由を聞くと、「マップが広く、アイコンも多いため、誰を倒せばよいのか分からなかった」と話していました。

この経験から、広いマップや多くのアイコンから次の目的を判断することは、ゲームに慣れていないプレイヤーにとって負担になる場合があるのではないかと考えました。特に、どこへ行けばよいか分からないという迷いが強くなると、本来楽しんでほしい移動や戦闘そのものに集中しにくくなります。

そこで本作では、マップを見なくても探索範囲を把握しやすいステージ構造として、球体地形を採用しました。球体であれば一方向へ進み続けてもいずれ元の場所へ戻ることができ、遠くへ行きすぎて現在地が分からなくなる状況を減らせると考えました。

また、惑星の表面を一周できる構造にすることで、まだ見ていない裏側を探してみようという探索の動機も作れると考えました。分かりやすさのために探索を単純化するのではなく、惑星の表・側面・裏側そのものを探索対象にすることで、迷いにくさと探索する楽しさの両立を目指しています。

実際に複数の方にプレイしてもらったところ、大きく迷ってゲームを進められなくなるケースは見られず、「ステージが分かりやすい」という意見も得られました。プレイ人数やステージ構成による影響もあるため、球体地形だけの効果とは断定できませんが、現在もプレイテストを行いながら導線やステージ構成を調整しています。

2-2. 本作で届けたい体験と遊びの設計

本作では、2-1 で述べたゲームに慣れていない人でも迷いにくい体験を土台にしながら、分かりやすさだけで遊びを単調にしないことを重視しています。初心者や子どもでも進め方を理解しやすく、それでいてゲームに慣れた人には探索や戦闘で工夫する余地があるゲームを目指しています。

探索では、広いマップや多数のアイコンから目的地を探すのではなく、惑星の表・側面・裏側そのものを探索範囲として利用しています。さらに、2 体のスライムへの分身を取り入れ、片方でスイッチを操作してもう片方の進路を作るなど、「どこへ行けばよいか」ではなく「どうすれば進めるか」を考える遊びにつなげています。

戦闘では、基本となる攻撃操作は分かりやすくしつつ、連続攻撃、強攻撃、ガード破壊、打ち上げ、空中攻撃、ため攻撃などを組み合わせられるようにしています。単に操作を難しくするのではなく、慣れてきたプレイヤーほど攻撃をつなげたり、状況に応じて技を選んだりできる構成にすることで、遊び始めやすさと上達する楽しさの両立を目指しています。

また、アクションが苦手な人でも遊びやすい「らくらくスタイル」を用意するとともに、2 人プレイにも対応しています。1 人で遊ぶ場合は 2 体を切り替えながら攻略し、2 人で遊ぶ場合はそれぞれが 1 体ずつ操作することで、家族や友人と相談しながら一緒にステージを攻略する体験を作っています。

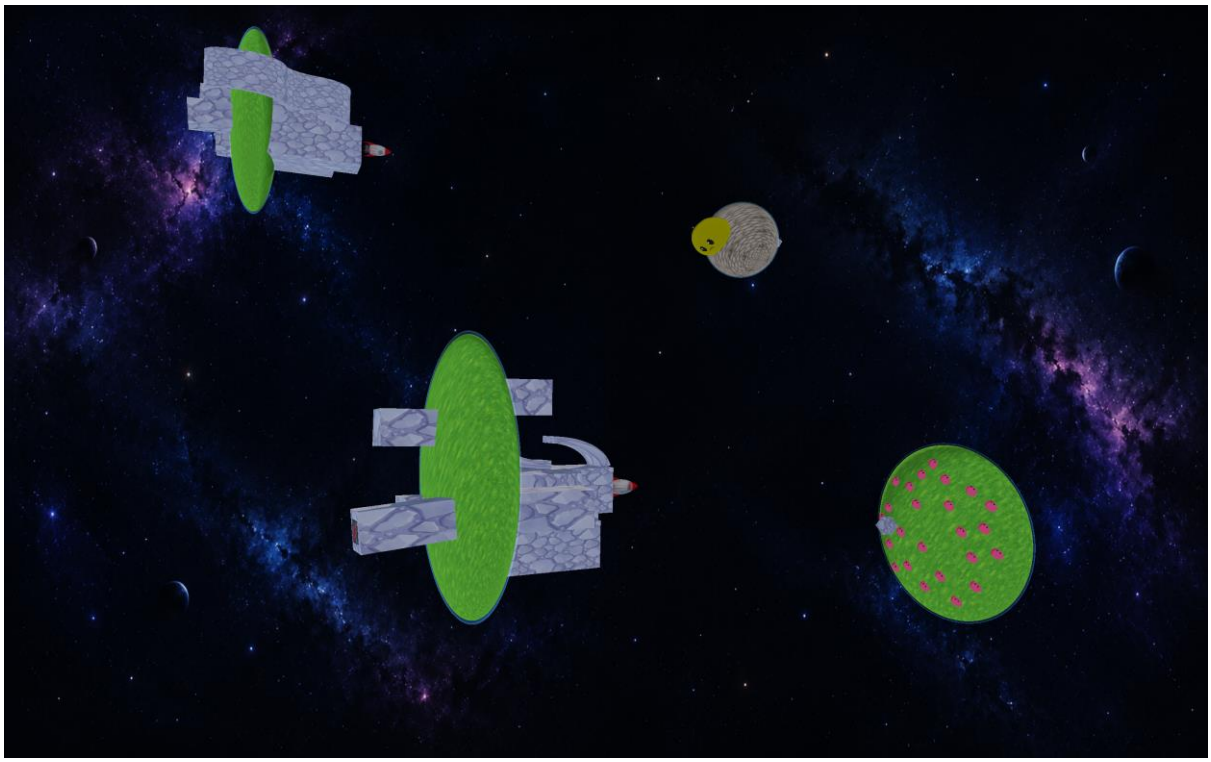
3. ゲームの特徴

3-1. 球体・楕円体の表裏まで探索できる 360° ステージ

本作では、一般的な平面地形ではなく、球体や楕円状の惑星をステージとして採用しています。

プレイヤーは惑星の表面だけでなく、側面や裏側まで自由に移動できます。惑星から突き出した足場や傾斜した地形にも沿って移動できるため、惑星全体を一つの立体的なステージとして探索できます。

「まだ見ていない裏側には何があるのか」を探しながら惑星を一周する探索体験が、本作の基本となる特徴です。



3-2. 2 体のスライムへの分身を使ったステージ攻略

プレイヤーは 1 体のスライムから 2 体へ分身し、操作するスライムを切り替えながらステージを攻略できます。

例えば、片方のスライムでスイッチを押している間に、もう片方のスライムで出現した足場を進むなど、2 体を別々の場所で行動させるギミックを用意しています。

球体ステージの表側と裏側に 2 体を分けて行動させることで、単に目的地を探すのではなく、「どちらをどこへ動かせば進めるか」を考える遊びにつなげています。

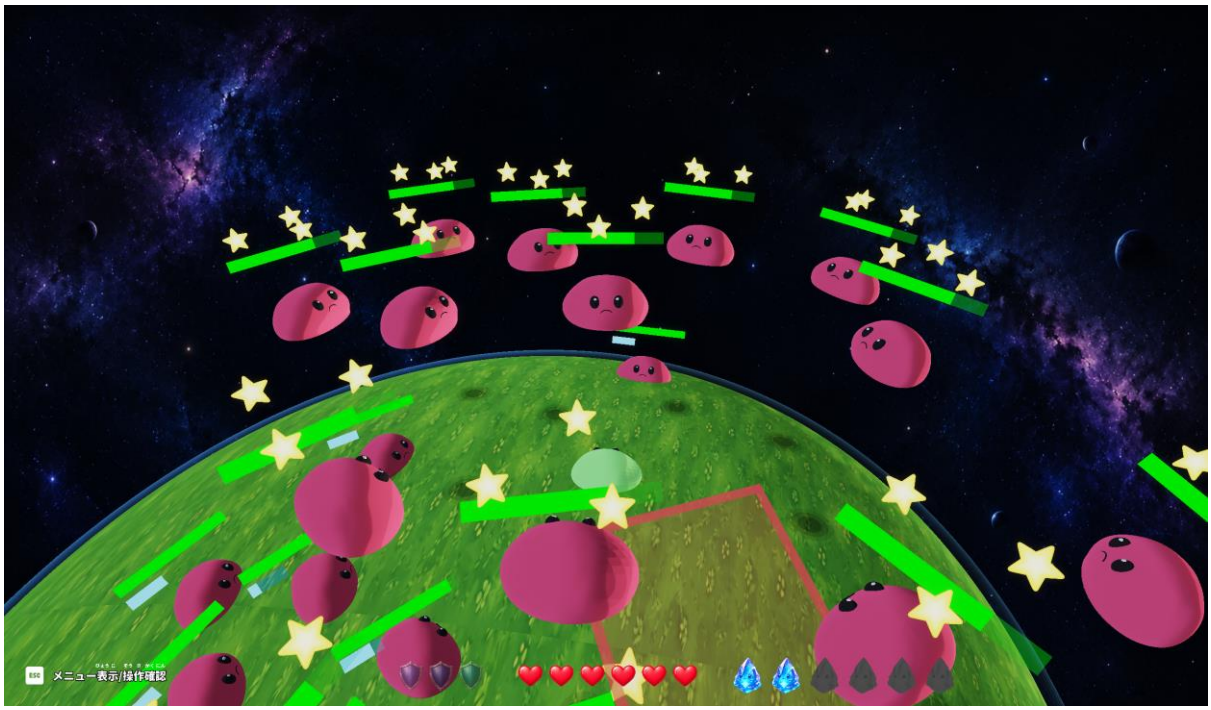


3-3. コンボ・打ち上げ・空中攻撃をつなげる戦闘

戦闘では、弱攻撃や強攻撃を連続して当てるコンボに加え、敵のガードを破壊して空中へ打ち上げ、そのまま空中攻撃へつなげることができます。

空中では、攻撃するたびに敵がより高いところに持ち上げられ、最後は高所から敵全体を惑星に叩きつける、爽快なアクションを行うことができます。

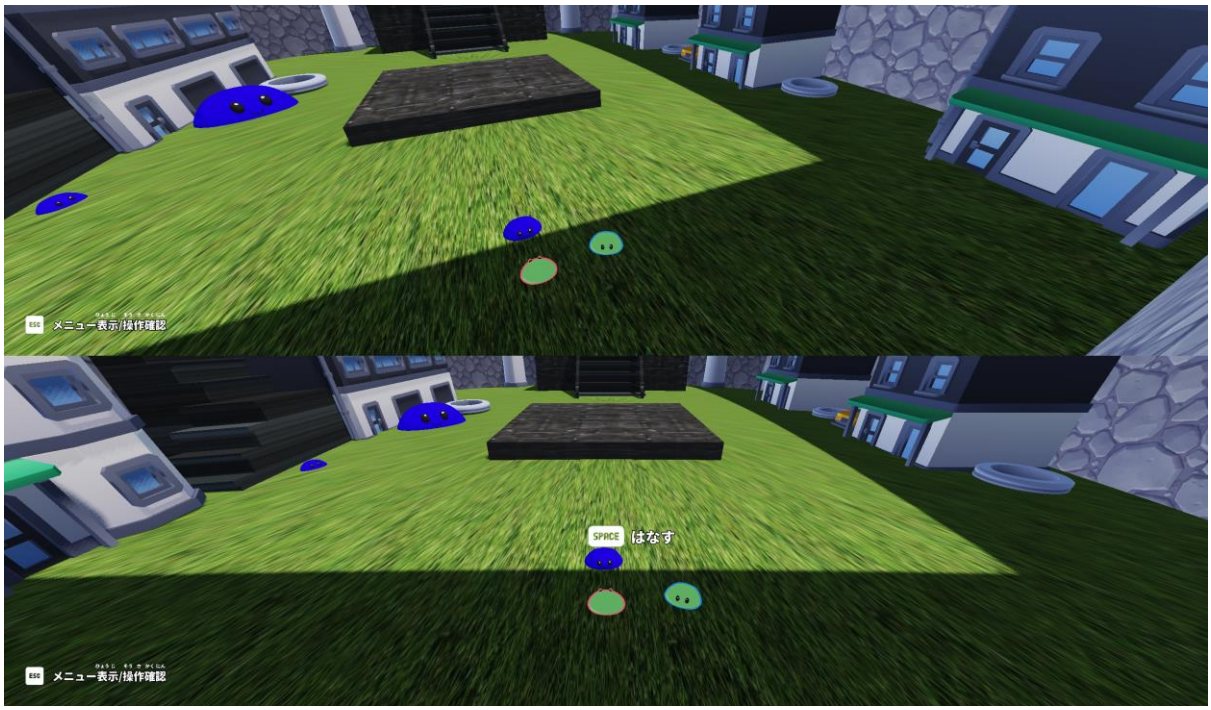
基本操作は分かりやすくしながら、操作に慣れるほど攻撃をつなげたり、より大きなダメージを狙ったりできる戦闘を目指しています。



3-4. 1 人でも 2 人でも遊べるプレイヤーシステム

本作は 1 人プレイと 2 人プレイの両方に対応しています。1 人プレイでは、分身した 2 体のスライムを切り替えながら操作します。2 人プレイでは、それぞれのプレイヤーが 1 体ずつスライムを操作できます。

同じステージや分身ギミックを、1 人では操作対象を切り替えながら、2 人では声をかけ合いながら攻略できるようにすることで、遊ぶ人数によって異なる楽しさが生まれるようにしています。



3-5. アクションが苦手な人向けの「らくらくスタイル」

アクションゲームに慣れていない人でも遊びやすいように、「らくらくスタイル」を用意しています。

通常の操作でアクションを楽しみたいプレイヤーだけでなく、複雑な操作やタイミングの入力が苦手な人でもステージ探索や戦闘を楽しめることを目指しています。

本作では、難しくすることで遊びの深さを作るだけではなく、プレイヤーに合わせて遊び方を選べることも重視しています。



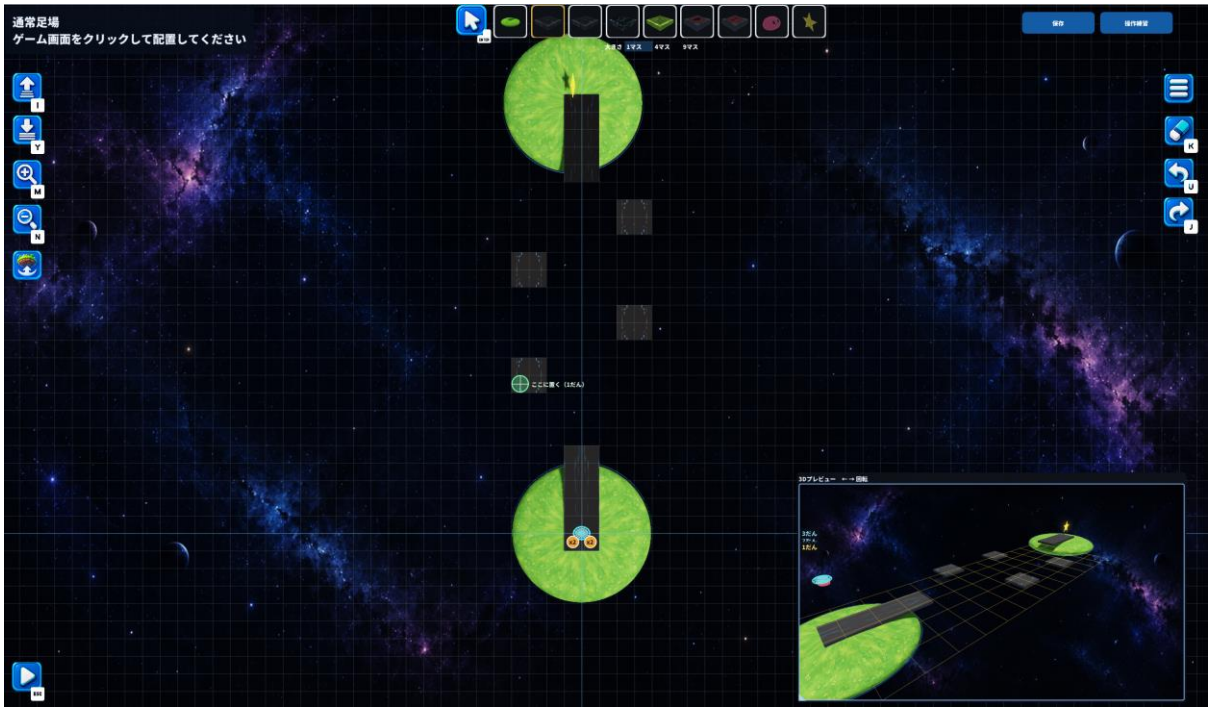
3-6. 2D グリッドと 3D プレビューを組み合わせたステージ作成機能

本作では、プレイヤー自身がオリジナルステージを作成し、そのまま遊べるステージ作成機能を実装しています。

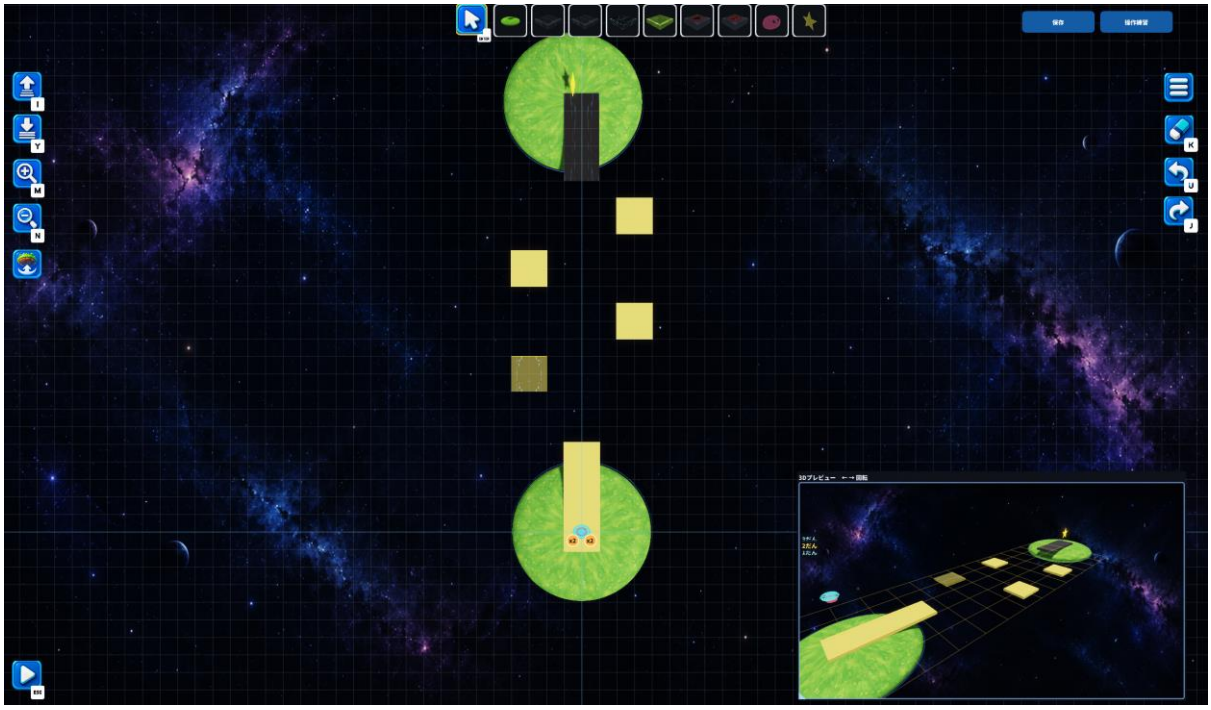
ステージは 2D グリッド上にオブジェクトを配置して作成します。3D 空間の高さは「だん」として管理し、編集する高さを切り替えながらオブジェクトを配置することで、3D 座標を直接入力しなくても立体的なステージを作成できます。

また、編集中のステージは 3D プレビューで確認できるため、2D 上での配置と実際のゲーム内での見え方を行き来しながら調整できます。作成したステージは保存でき、完成後はそのステージを実際にプレイして攻略できます。

3D 編集の複雑さを抑えながら、球体ステージならではの立体的な配置を作れることを重視して設計しています。



長押しで塗り絵をする感覚で足場を配置



足場の高さを「だん」で管理



作成したステージで遊ぶことができる

3-7. 操作方法

ゲームコントローラーの場合（開発者推奨）

L スティック: 移動

R スティック: カメラ回転

A: ジャンプ・決定

Y: 弱攻撃

X: 強攻撃

B: ダッシュ

Y 長押し: 連続攻撃（ジュエルを使用）

X 長押し: ため攻撃（ジュエルを使用）

L: 回復（ジュエルを使用）

ZR: 分身 / 合体

ZL: 分身切り替え

R: カメラをプレイヤーの背後へ移動

キーボードの場合

W / A / S / D: 移動

矢印キー: カメラ回転

SPACE: ジャンプ

J: 弱攻撃

K: 強攻撃

U: ダッシュ

J 長押し: 連続攻撃（ジュエルを使用）

K 長押し: ため攻撃（ジュエルを使用）

N: 回復（ジュエルを使用）

I: 分身 / 合体

Y: 分身切り替え

M: カメラをプレイヤーの背後へ移動

（P: デバッグエディタ表示切り替え、L: 自由カメラモード切り替え、自由カメラモード時は WASD で移動、矢印キーでカメラを回転できます。）

4. 技術概要

4-1. 使用技術（言語・ライブラリ）

プログラミング言語: C++20 / GLSL

グラフィックス API: OpenGL

ウィンドウ管理・キーボード入力: GLFW

ゲームコントローラー入力: SDL2

音声再生: SDL_mixer

フォント読み込み: SDL_ttf

物理演算・衝突判定: Bullet Physics

3D モデル・ボーンアニメーション読み込み: Assimp

データ読み込み: yaml-cpp

数学・ベクトル・行列演算: GLM

開発用 UI・3D ギズモ: Dear ImGui / ImGuizmo

ビルド・依存関係管理: CMake / vcpkg

※Unity や Unreal Engine などの汎用ゲームエンジンは使用していません。

OpenGL を中心に、描画、キャラクター制御、カメラ、物理との連携、 アニメーション、ステージ管理、UI、各種開発用エディタなど、ゲームを構成する主要なシステムを実装しています。

4-2. 主な技術選定理由

C++ / OpenGL

ゲームエンジンが提供する機能を利用するだけでなく、描画、キャラクター制御、 カメラ、物理との連携など、3D ゲームを構成する仕組みを自分で実装しながら 理解することを目的として、C++ / OpenGL を採用しました。

また、Windows と macOS の両方で動作させ、異なる環境でプレイテストを行える構成にしたいという目的があり、DirectX ではなく OpenGL を選択しています。

Bullet Physics

本作では、球体・楕円体上での接地や、惑星上に配置した足場との衝突など、 単純な平面地形とは異なる判定が必要になります。

そのため、Ray Test、Sweep Test、Contact Test などの衝突判定機能を利用できる Bullet Physics を採用しました。Bullet の判定結果を利用しながら、接地判定、壁との衝突処理、カメラ衝突など、本作独自の処理を実装しています。

YAML / yaml-cpp

ステージ上の Actor 配置、UI、敵 AI、カメラ、シーケンスなど、調整頻度の高いデータをソースコードから分離するために YAML を採用しました。

以前 JSON を使用した際に、調整中のデータをコメントアウトできないことを不便に感じた経験があり、コメントを記述できることに加えて、階層構造を持つデータも編集しやすい YAML を選択しました。

CMake / vcpkg

Windows と macOS の両方で同じプロジェクト構成を利用できるように、ビルド管

理には CMake を採用しました。

また、Bullet Physics、Assimp、SDL2 などの外部ライブラリは vcpkg で管理し、環境ごとの依存関係の差をできるだけ小さくしています。

5. 開発用エディタ・制作支援機能

本作では、ゲーム本体だけでなく、ステージ、UI、チュートリアル、演出、各種パラメータなどを効率よく調整するための開発用ツールも実装しています。

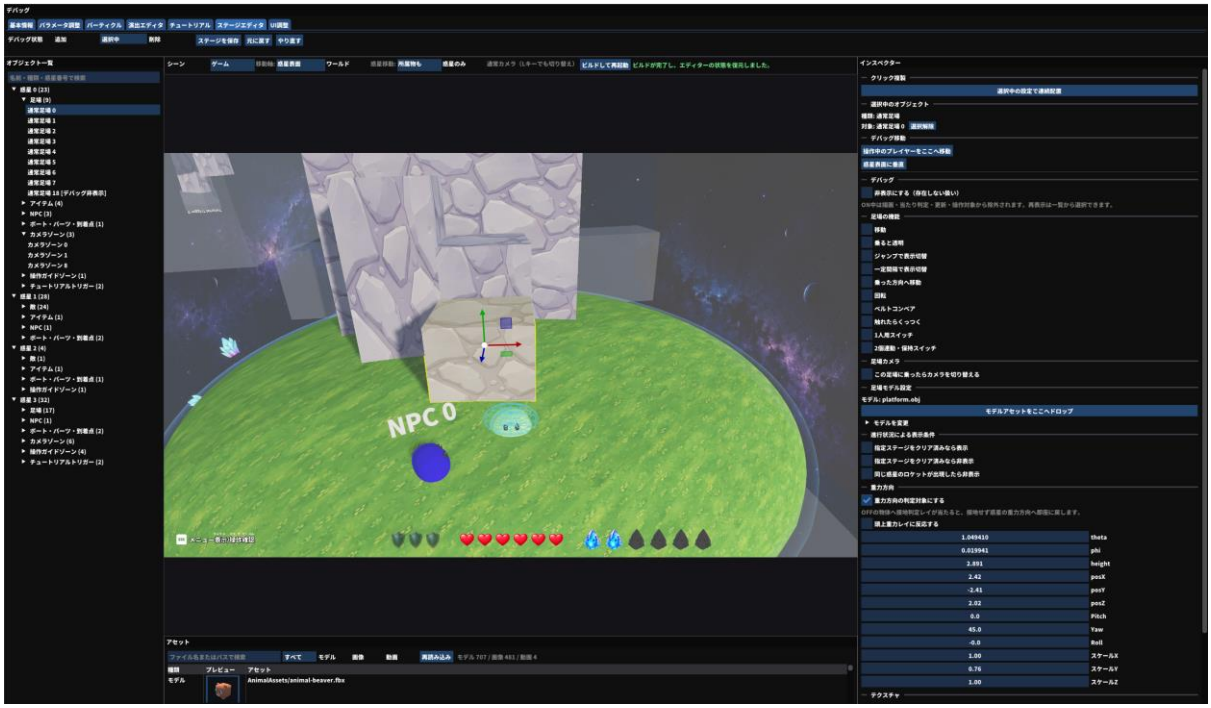
ソースコードを書き換えてビルドし直す作業をできるだけ減らし、実際のゲーム画面を確認しながら「調整 → 確認 → 修正」を繰り返せる開発環境を作ることを重視しました。

5-1. ゲーム内でステージを直接編集できる 3D ステージエディタ

ゲームを実行したまま、ステージ上の Actor を選択し、位置・回転・大きさなどを編集できる 3D ステージエディタを実装しています。

ゲーム画面上からオブジェクトを選択し、3D ギズモを使用して移動・回転・拡大縮小を行えるほか、Actor の追加・複製・削除などにも対応しています。編集結果をデータへ反映できるため、座標を数値で入力して起動し直すことなく、実際の見え方を確認しながらステージを調整できます。

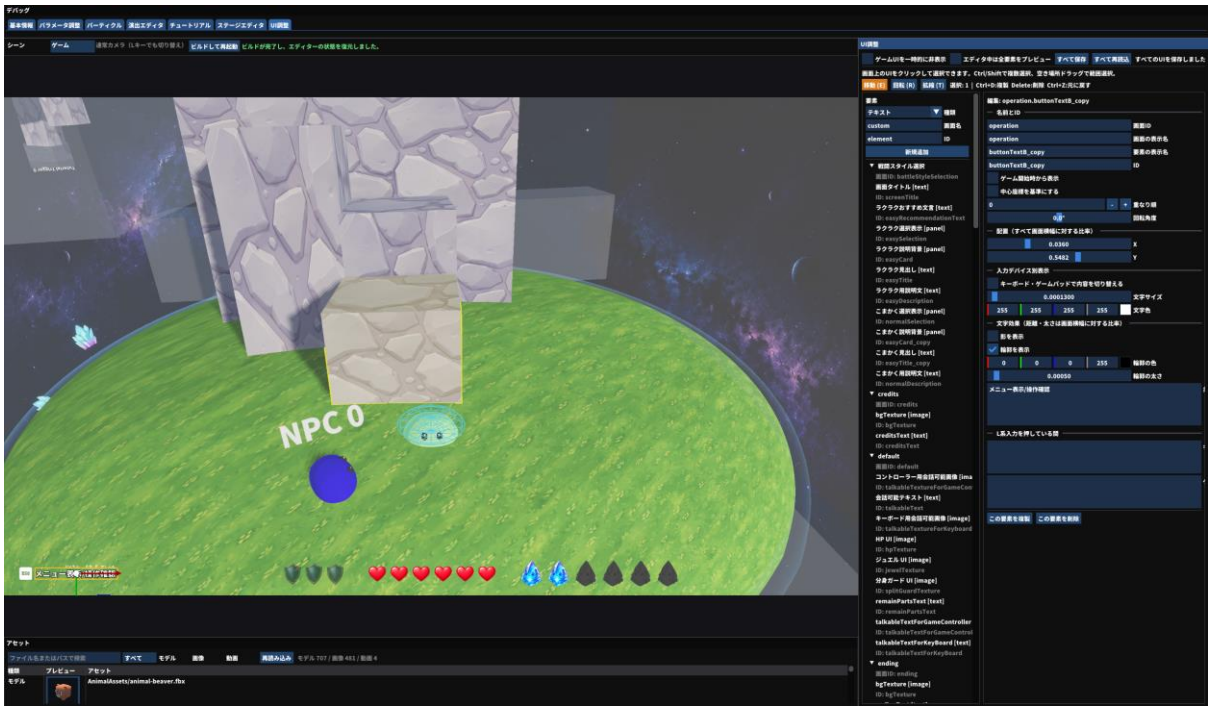
また、自由カメラを利用することで、通常のプレイヤーカメラでは確認しにくい場所からも配置や当たり判定を確認できます。



5-2. 実際のゲーム画面上で調整できるビジュアル UI エディタ

UI についても、設定ファイルの数値を直接変更するだけではなく、実際のゲーム画面を見ながら配置を調整できる UI エディタを実装しています。

テキスト・画像・パネルの追加に加えて、既存 UI の選択、移動、回転、拡大縮小、複製、削除などを画面上から行えます。ゲーム本編やメニュー、チュートリアルなどの実際の表示状態を確認しながら調整できるため、UI 制作時の試行錯誤を行いやすくしています。



5-3. チュートリアルをデータから構成・調整できる仕組み

本作では、チュートリアルの文章や進行内容をプログラム内に直接記述せず、データとして管理しています。

通常の文章に加えて、ゲームコントローラー用・キーボード用の説明、ルビ、目標表示、動画、注目させる対象、次のページへ進む条件などを組み合わせてチュートリアルを構成できます。

また、「らくらくスタイル」では通常とは異なる説明を表示できるようにし、操作方法が変わる場合でも同じチュートリアルの中で内容を切り替えられるようにしています。

これにより、チュートリアルの文章や順番を変更するためだけにプレイヤー処理を書き換える必要がなく、プレイテスト後の説明追加や調整を行いやすくしています。



5-4. カメラやイベント演出をデータから調整

会話やイベントなどで使用するカメラ演出についても、ゲームロジックへ個別に処理を書き込むのではなく、演出内容をデータとして管理できるようにしています。

カメラ位置や注視対象などを組み合わせて演出を構成し、ステージ進行や会話などから呼び出せるようにすることで、同じ仕組みを複数の場面で利用できるようにしています。

演出用データをゲームロジックから分離することで、カメラの見え方や演出時間を調整する際に、ステージ進行処理そのものへ変更を加える範囲を抑えています。



5-5. プレイヤー・敵・カメラなどのパラメータ調整

ゲームバランスや操作感を調整するため、プレイヤー、敵、カメラなどの各種パラメータをソースコードへ固定せず、外部データとして管理しています。

移動や攻撃に関する値、カメラの距離・感度、敵の行動に関する値などをデータ側から変更できるため、プログラムの構造を変更せずにゲームの手触りを繰り返し調整できます。

また、開発用 UI から値を確認・変更できる項目も用意し、実際に操作した結果を見ながらパラメータを調整できるようにしています。



5-6. モデルやテクスチャなどを確認するアセット表示機能

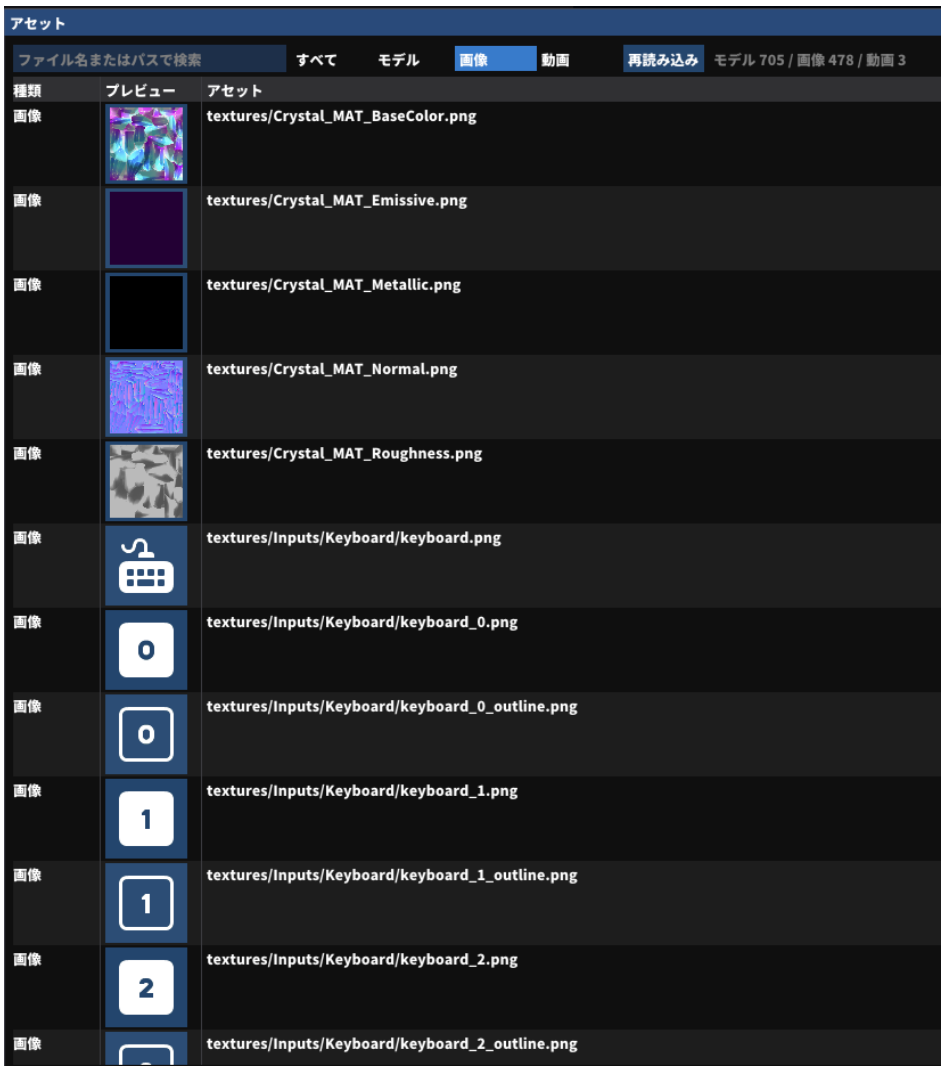
開発中に使用する 3D モデルやテクスチャを一覧から確認・選択できるアセット表示機能を用意しています。

ファイル名やパスだけで判断するのではなく、アセットの見た目を確認してから Actor や UI へ設定できるため、ステージや UI を編集する際に目的のアセットを探しやすくしています。

アセット

ファイル名またはパスで検索　すべて　モデル　画像　動画　再読み込み　モデル 705 / 画像 478 / 動画 3

種類	プレビュー	アセット
モデル		AnimalAssets/animal-beaver.fbx
モデル		AnimalAssets/animal-bee.fbx
モデル		AnimalAssets/animal-bunny.fbx
モデル		AnimalAssets/animal-cat.fbx
モデル		AnimalAssets/animal-caterpillar.fbx
モデル		AnimalAssets/animal-chick.fbx
モデル		AnimalAssets/animal-cow.fbx
モデル		AnimalAssets/animal-crab.fbx
モデル		AnimalAssets/animal-deer.fbx
モデル		AnimalAssets/animal-dog.fbx
モデル		AnimalAssets/animal-elephant.fbx
モデル		AnimalAssets/animal-fish.fbx



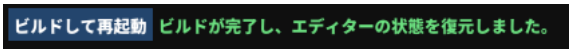


5-7. 編集状態を保持したビルド・再起動

C++コードを変更した場合でも確認作業を続けやすいよう、開発用エディタからビルドとゲームの再起動を実行できる機能を用意しています。

再起動前のステージやエディタの状態を保持し、起動後に復元することで、コー

ド修正後もできるだけ同じ状態から確認作業を再開できるようにしています。

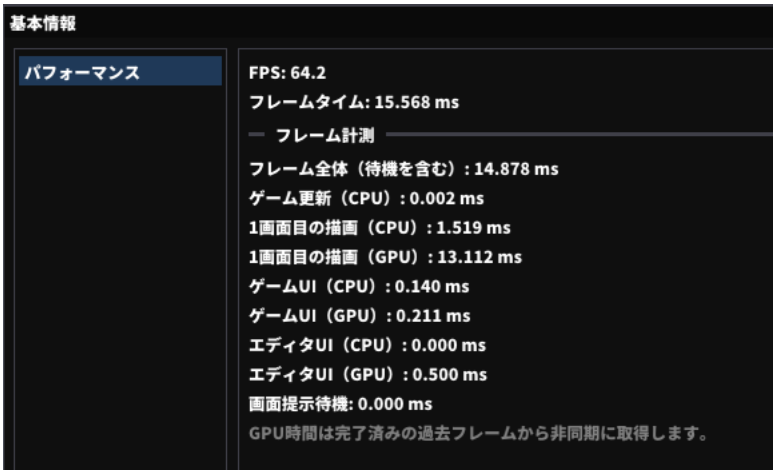


5-8. パフォーマンス計測・デバッグ支援

描画負荷の原因を確認するため、OpenGL の Timer Query を利用した GPU 処理時間の計測機能を実装しています。

描画処理ごとの負荷を数値として確認できるようにすることで、「画面が重い」という感覚だけではなく、どの描画処理に時間がかかっているのかを調査しながら最適化できるようにしています。

そのほかにも自由カメラや各種デバッグ表示などを用意し、通常プレイでは確認しにくい状態を開発中に確認できるようにしています。



6. システム構成・設計について

6-1. 継承・Component・専用クラスの使い分け

本作では、すべての機能を継承だけで表現するのではなく、機能の性質に応じて継承、Component、専用クラスを使い分けています。

Actor として固有の意味や振る舞いを持つものは Actor を継承したクラスとして

実装し、「取得できる」「破壊できる」「一定時間カメラを向ける」といった複数の Actor で再利用できる機能は Component として分離しています。

例えば、収集機能を CollectableComponent、破壊可能な機能を DestructibleComponent、注目対象となる機能を FocusComponent として分離することで、それぞれの Actor に必要な機能を組み合わせられる構成にしています。

一方で、すべてを Component にすると Component 同士の依存関係が複雑になるため、特定の Actor だけで使用する処理や、複数の処理をまとめて一つの責務を担う処理については専用クラスとして分離しています。

このように、1 つの方式に統一するのではなく、機能の再利用性や責務に応じて構成方法を使い分けています。

6-2. Player・Enemy の責務分割

開発初期は、Player や Enemy のクラスに移動、攻撃、状態管理など多くの処理を直接持たせていました。しかし機能追加を続けるにつれてクラスが肥大化し、一つの機能を変更した際に他の処理へ影響しやすくなりました。

そこで、Player では入力、移動、接地、惑星重力、戦闘、HP などの状態、状態遷移、リスポーン、インタラクション、アニメーション、エフェクトなどを、それぞれ専用クラスへ分離しました。

Player 本体はこれらを保持し、必要な処理を各クラスへ委譲することで、移動を変更する場合は PlayerMovement、攻撃を変更する場合は PlayerCombat のように、変更箇所を判断しやすい構成にしています。

Enemy についても同様に、状態管理、状態遷移、移動、戦闘、ダメージ処理、行動 AI をそれぞれ分離しています。

これにより、Player や Enemy というゲーム上のまとまりは維持しながら、内部の責務を小さく分け、機能追加や修正を行いやすくしています。

6-3. ゲーム全体のシステム分割

Player や Enemy だけでなく、ゲーム全体についても、一つの Game クラスへすべての処理を集中させないように責務を分離しています。

物理演算は PhysicsSystem、カメラは CameraSystem、入力は InputSystem、シー

ン管理は SceneSystem、3D 描画は Renderer3D、UI 描画は UIRenderer、演出進行は SequenceSystem など、それぞれの役割を担当するシステムとして分離しています。

また、ステージ進行、ゲーム進行状況、UGC モード、UGC プレビュー、デバッグエディタのセッション管理など、ゲーム固有の処理についても専用の Controller へ分離しています。

Game クラスは各システムを生成・保持し、ゲーム全体の処理をつなぐ役割を担当します。各処理の詳細をそれぞれのシステムへ委譲することで、機能ごとの責任範囲を明確にしています。

6-4. リファクタリングで重視したこと

本作は機能追加を続けながら開発してきたため、実装後も定期的にコード構成を見直し、責務の分離や命名の改善を行っています。

特に、クラスや関数が複数の役割を持ち始めた場合には、その処理を専用クラスや関数へ分離し、変更理由が異なる処理をできるだけ同じ場所へ集めないことを意識しています。

また、関数名から処理の目的を判断できるようにすること、複雑な条件には意味の分かる名前を付けること、変更しない値や関数には const、コンパイル時に決定できる値には constexpr を使用することなどをコーディング規約として定めています。

個人開発であっても、数か月後に自分自身が読み返した場合や、第三者がコードを確認した場合に、処理の役割を追いやすいコードを目指しています。

7. 技術的に工夫した点・問題解決

7-1. 球体・楕円体上で安定して移動するための接地・重力制御

本作では、球体・楕円体の表面だけでなく、惑星から突き出した足場や斜面も連続して移動するため、ワールド座標の Y 方向を常に「上」として扱うことができません。当初は惑星中心からプレイヤーへの方向を上方向としていましたが、平らな足場や斜面では実際の接地面と向きが一致せず、姿勢が不自然になる問題がありました。

そこで、Bullet Physics の Ray Test をプレイヤーの中央・前後左右の 5 か所から行い、取得した地面法線から上方向を決定しています。中央の法線には 3 倍の重みを与えて平均し、周囲の Ray が中央とは別の Collision Object に当たった場合は平均から除外することで、足場の端などで姿勢が急変することを抑えています。

また、楕円体では惑星中心方向と表面法線が一致しないため、各軸の半径とプレイヤー位置から楕円体表面上の近接点を求め、その位置の外向き法線を計算しています。楕円体外部の点については、表面上の近接点を求めるための値を二分探索で算出しています。空中では周囲の惑星との表面距離から重力対象を判断し、見た目の上方向は補間して変化させることで、ジャンプや惑星間移動でも急激に姿勢が変わらないようにしています。

これらを組み合わせることで、単純な球体だけでなく、楕円体・斜面・足場・惑星間を含む地形でも安定して移動できるようにしました。

7-2. 3D ステージ編集を 2D グリッドと「だん」に置き換えた UGC システム

本作ではプレイヤー自身が 3D ステージを作成できますが、一般的な 3D エディタのように X・Y・Z 軸を自由に操作する方式では、3D 編集に慣れていないプレイヤーには位置関係を把握しにくいと考えました。

そこで、水平方向を 2D グリッド、高さ方向を「だん」として分離しました。内部ではグリッド 1 マスの大きさを gridSize として統一し、ドラッグによる移動量を round(移動量 / gridSize) × gridSize で丸めることで、Actor が常にグリッド単位で移動するようにしています。高さも整数の編集レイヤーとして管理し、段を変更すると Y 座標を gridSize 分移動させます。既存 Actor を選択した場合は、Y 座標から対応する「だん」を逆算して編集レイヤーを同期します。

また、マウス位置から 3D 空間へ Ray を飛ばし、選択 Actor を通る仮想平面との交点からドラッグ先を計算しています。削除時も、現在の「だん」にある Actor だけを消す処理や、UGC 用足場を 1 マス単位で消す処理を分けて実装しています。Undo / Redo にも対応しているため、配置を試しながら元へ戻すこともできます。

これにより、内部では 3D 座標を扱いながら、プレイヤーは「1 マス動かす」「1 だ

ん上げる」といった分かりやすい操作で立体的なステージを作れるようにしました。

7-3. UI 内の漢字へ自動でルビを表示する仕組み

本作では子どもでも文章を読みやすくするため、UI や会話文に含まれる漢字へルビを表示しています。しかし、漢字ごとにルビ用 UI を手作業で配置すると、文章を追加するたびに作業が発生し、本文や文字サイズを変更した際には位置調整もやり直す必要があります。

そこで Windows 版では、JapanesePhoneticAnalyzer を利用して文章から読みを自動取得する仕組みを実装しました。文字の Unicode 範囲から漢字・ひらがな・カタカナを判別し、日本語部分を解析したうえで、DisplayText と YomiText から「本文」「読み」「ルビを表示するか」を持つ RubyTextSegment へ変換します。漢字を含む Segment だけに読みを付与するため、ひらがな部分へ不要なルビは表示されません。

また、解析結果の本文を再結合し、元の文章と一致するかを確認してから利用することで、解析結果と文字位置がずれた状態でルビを表示しないようにしています。描画側には通常の文字描画とは別に DrawRubyText を用意し、一度生成したルビ情報はキャッシュして再利用しています。

これにより、文章ごとにルビ用 UI を作成・配置する必要がなくなり、通常の文章を登録するだけで漢字へルビを表示できるようにしました。

7-4. UI をゲーム画面上で追加・配置できるビジュアルエディタ

当初、UI の位置やサイズは YAML 内の数値を書き換えて調整していました。しかし、「少し右へ動かす」といった視覚的な修正でも、数値変更と再読み込みを何度も繰り返す必要があり、UI が増えるほど調整に時間がかかっていました。

そこで、実際のゲーム画面を編集キャンバスとして使用する UI エディタを実装しました。エディタ上からテキスト・画像・パネルを新規追加でき、追加された要素は UILoadSystem へ登録されます。位置やサイズは固定ピクセルではなく画面サイズに対する比率として保持し、エディタで編集した値をそのままゲーム本編の UI データ

として使用します。

画面上へ描画した UI の位置・大きさ・回転角度を記録しておき、マウス座標との当たり判定から直接 UI を選択します。選択後は ImGuizmo を使って移動・回転・拡大縮小でき、複数選択、範囲選択、複製、削除、Undo にも対応しています。

これにより、「数値を変更して結果を見る」方式から、「実際の画面を見ながらその場で配置する」方式へ変更でき、新規 UI の作成から位置調整までをエディタ内で行えるようにしました。

7-5. 3D モデルを自動撮影して生成するアセットサムネイル

3D モデルの数が増えるにつれて、ファイル名やパスだけではモデルの見た目を判断しにくくなり、目的のモデルを確認するために一度 Actor へ設定してゲーム内へ表示する必要がありました。

そこで、アセットブラウザに表示する 3D モデルをプログラム側で自動撮影し、サムネイル Texture として生成する仕組みを実装しました。256×256 の Texture と専用 Framebuffer・Depth Buffer を作成し、ゲーム画面とは別にモデルをオフスクリーン描画しています。モデルごとに大きさが異なるため、Bounding Box からモデル中心と半径を求め、半径と画角からカメラまでの距離を自動計算することで、大きさの異なるモデルも一定の見え方になるようにしています。

生成結果は「モデルパス・スケール・差し替えテクスチャ」を組み合わせたキーでキャッシュし、同じモデルを何度も描画しないようにしています。また、一度に大量生成して処理が重くならないよう、新規サムネイルの生成は 1 フレームに 1 枚へ制限しています。サムネイル描画前の Framebuffer や Viewport、Depth Test などの OpenGL 状態も保存し、処理後に復元することでゲーム本編の描画へ影響しないようにしました。

これにより、モデルを実際にステージへ配置しなくても、アセット一覧の画像を見ながら目的のモデルを選択できるようになりました。

7-6. 編集状態を保持したビルド・再起動システム

開発用エディタによって多くの調整はゲームを起動したまま行えますが、C++コード自体を変更した場合には再ビルドが必要です。そのたびにゲームを終了し、再起動後に同じステージへ移動してエディタを開き直す必要があり、細かな修正を繰り返す際の負担になっていました。

そこで、エディタから「現在の編集状態を保存 → ビルド → ゲームを再起動 → 編集状態を復元」まで行える仕組みを実装しました。ビルド要求時には、現在のエディタ状態を YAML へ保存した後、ゲーム本体とは別の Build Restart Helper Process を起動し、ゲーム本体を終了します。Windows では CreateProcessW、macOS では fork / execl を用いて Helper を起動し、現在の Build Directory や実行ファイル、Session File などを引数として渡します。

再ビルド後に新しい実行ファイルが起動すると、保存していた Session File を読み込み、エディタの状態を復元します。ビルドに失敗した場合は Build Log の場所をエディタ上へ表示するようにしています。また、実行中のゲーム自身から Executable Path を取得することで、環境ごとに固定パスを記述しない構成にしています。

これにより、C++コードを変更した場合でも「問題を確認 → コードを修正 → 同じ状態で再確認」という反復作業に必要な手順を減らすことができました。

8. ビルド・配布

本作では、自分の開発環境だけで動作する状態ではなく、Windows / macOS の両方でビルド・実行でき、他の方にもプレイしてもらえる状態を目指しました。

ビルドには CMake を使用し、外部ライブラリは vcpkg を利用して管理しています。開発環境固有の設定への依存を減らし、Windows / macOS で同じプロジェクト構成を利用できるようにしています。

配布時には、実行ファイル、必要なライブラリ、assets、shaders などをまとめた配布用フォルダを作成し、Windows 版・macOS 版を GitHub Releases で公開しています。

また、一般のプレイヤーにも作品を知ってもらい、遊んでもらえるように itch.io でも作品ページを公開しています。

GitHub Releases

https://github.com/kusaka0914/SlimesSpaceTravel/releases

itch.io

https://kusaka0914.itch.io/slimesspacetravel

これにより、自分の開発環境以外でも起動確認やプレイテストを行えるようにし、実際のプレイを通して不具合や操作性の問題を確認しています。

プレイテスト後にはGoogle Formsで感想を収集し、得られた意見をもとに操作性やゲーム内容の改善を行っています。

9. まとめ

9-1. 制作を通して得たこと

本作の制作を通して最も理解が深まったのは、3Dゲームを構成する各システムのつながりと、長期間開発を続けるための設計の重要性です。

C++ / OpenGLを用いて汎用ゲームエンジンに頼らず開発したことで、画面に3Dモデルを表示するだけでなく、入力、物理、カメラ、アニメーション、UIなどの処理がどのように連携して一つのゲームとして動作しているのかを、実装を通して理解することができました。

また、開発初期にはPlayerやEnemy、Gameクラスへ処理を追加していく方法でも開発を進められましたが、規模が大きくなるにつれて、機能追加や修正の影響範囲を把握しにくくなる問題を経験しました。この経験から、単にコードを動かすだけではなく、役割の異なる処理を分離し、後から変更しやすい構成を考えることの重要性を学びました。現在では、新しい機能を追加する際にも「どのクラスに書くか」だけでなく、「どの責務として分離するべきか」を意識するようになっています。

さらに、ゲームの品質を高めるには、ゲーム本体の実装だけでなく、試行錯誤を繰り返しやすい開発環境も重要だと感じました。ステージやUI、パラメータを素早

く調整し、パフォーマンスを確認できる環境を整えることで、実装そのものだけでなく、その後の調整や検証まで含めて開発を設計する視点を得ることができました。

9-2. 今後の展望

本作については、今後もプレイテストを重ねながら、戦闘演出、敵の行動バリエーション、ステージ構成などを調整し、操作したときの気持ちよさと遊びやすさを高めていきたいと考えています。

また、現在実装しているステージ作成機能や開発用エディタについても拡張を続け、より少ない操作でステージや演出を作成・調整できる制作環境を目指します。ゲーム本体だけでなく、制作を支える仕組みも継続して改善していきたいです。

さらに、現在は作成したステージをローカルで保存・プレイする形式ですが、将来的にはネットワーク機能を導入し、他のプレイヤーが作成したステージを共有して遊べる仕組みに発展させることも考えています。

本作で培った 3D アクション、物理制御、データ駆動、エディタ開発の経験を、今後のゲーム開発でも活かし、より規模の大きな作品でも継続して開発・調整しやすい環境を構築できるようにしたいと考えています。